Memory Notes

Simplified RV32I: kestrel Simplified edition - the single-cycle RV32I core, rung 1 of the falcon suite

October 23, 2026

Table of Contents

1 Simplified Rv32I Index....11

2 Introduction....12

2.1 What this book is....12

2.2 What kestrel is....13

2.3 Why single-cycle comes first....14

2.4 How to read this book....15

3 The Falcon Ladder....16

3.1 The suite....16

3.1.1 Figure 1.1: The falcon ladder, smallest to largest....17

3.2 What each rung teaches....18

3.3 Why kestrel is rung 1....19

4 Conventions....20

4.1 Numbers and identifiers....20

4.2 Register names....21

4.3 Halt versus trap....22

4.4 RVFI....23

4.5 Citations....24

5 Instruction Formats....25

5.1 Six formats, one word....25

5.2 How kestrel decodes a format....26

5.3 Immediate assembly....27

5.4 Shift immediates....28
6 The Instruction Set....29
6.1 The thirty-seven....29
6.1.1 Integer register-register (R-type)....29
6.1.2 Integer register-immediate (I-type)....29
6.1.3 Loads and stores....30
6.1.4 Branches....31
6.1.5 Jumps and upper immediates....31
6.2 Reserved encodings....33
7 The System Layer: Stubs, NOPs, and Halts....34
7.1 What RV32I surrounds itself with....34
7.2 FENCE and FENCE.I retire as NOPs....35
7.3 ECALL and EBREAK halt....36
7.4 MRET falls through....37
7.5 The CSR class retires as a bounded stub....38
7.6 Halt causes, collected....39
8 Datapath Overview....40
8.1 The machine in one picture....40
8.1.1 Figure 3.1: kestrel block diagram....40
8.2 The module list....41
8.3 The state inventory....42
8.4 Reading the chapters that follow....43
9 One Cycle, End to End....44

9.1 The clock in one sentence....44
9.2 Fetch and decode....45
9.3 Operands and immediates....46
9.4 Execute....47
9.5 Next-PC selection....48
9.6 Memory access....49
9.7 Writeback....50
9.8 Two worked examples....51
10 Control State....52
10.1 Why there is no FSM....52
10.2 The halt holding register....53
10.3 No retry bit, no merge register....54
10.4 The retirement counter....55
10.5 What this buys....56
11 The RVFI Retire Interface....57
11.1 Retirement as the observation channel....57
11.2 The signal table....58
11.3 The three gating rules....60
11.4 Memory-field packing....61
11.5 Why this pays....62
12 The Control Bundle....63
12.1 Thirteen wires out of decode....63
12.2 The key: opcode, funct3, funct7....65

12.3 The default bundle.....66
12.4 Reading the table.....67
13 The Decode Table.....68
13.1 Arithmetic and upper-immediate rows.....69
13.2 Loads, stores, branches, jumps.....70
13.3 System, fence, and the default.....71
13.4 What the table teaches.....72
14 The Memory Contract.....73
14.1 The interface.....73
14.2 Timing: one cycle, except when documented.....75
14.3 What the contract deliberately excludes.....76
15 Misaligned Data Accesses.....77
15.1 What the spec permits.....77
15.2 The rotation model.....78
15.3 The misaligned halt.....79
15.4 Why not hardware?.....80
16 IALIGN and the Misaligned-Target Halt.....81
16.1 The rule.....81
16.2 Why decode cannot catch it.....82
16.3 The writeback subtlety.....83
16.4 The six halt causes, in context.....84
16.5 The fence_i connection.....85
17 Testbench and Golden Model.....86

17.1 The harness....86
17.2 The golden interpreter....87
17.3 Programs and the hex pipeline....88
17.4 The run grammar....89
18 The rv32ui Battery....90
18.1 What the battery is....90
18.2 The verdict protocol....93
18.3 What the battery proves — and its seam....94
19 Spike Lockstep....95
19.1 What lockstep adds....95
19.2 The mechanics....96
19.3 ma_data, an expected halt....97
19.4 The mnstatus divergence....98
20 The riscv-formal Flow....99
20.1 What it is....99
20.2 kestrel's setup....100
20.3 The checks and depths....101
20.4 The flow's three honest workarounds....102
20.5 Results....103
21 The CEX Story: What Formal Caught That Forty-Two Tests Missed....104
21.1 The failure....105
21.2 Why the battery missed it....106
21.3 The fix....107

21.4 Aftermath....108
21.5 The lesson, stated once....109
22 What kestrel Costs....110
22.1 CPI 1, honestly stated....110
22.2 The critical path is the whole machine....111
22.3 The accounting, instruction by instruction....112
22.4 What would have to give....113
23 The Next Rung: merlin....114
23.1 The five-stage answer....114
23.1.1 Figure 7.1: kestrel’s cycle, cut into merlin’s stages....114
23.2 What appears when the cycle is cut....115
23.3 What carries over unchanged....116
23.4 Why the ladder stops here first....117

List of Figures

Figure 1.1: The falcon ladder, smallest to largest.....17
Figure 3.1: kestrel block diagram.....40
Figure 7.1: kestrel's cycle, cut into merlin's stages.....113

List of Tables

Table 1: The falcon ladder....16
Table 2: The six RV32I instruction formats (spec Volume I, section 2.1.2)....25
Table 3: Immediate assembly per format (spec Volume I, section 2.1.3)....27
Table 4: OP group: ten register-register instructions....29
Table 5: OP-IMM group: nine register-immediate instructions....29
Table 6: LOAD and STORE groups: five loads, three stores....30
Table 7: BRANCH group: six conditional branches....31
Table 8: JAL, JALR, LUI, AUIPC....31
Table 9: The six halt causes (encoding centralized in common-rv32i/rtl/includes/rv32i_pkg.sv)....39
Table 10: kestrel's six modules....41
Table 11: The complete kestrel state inventory, register file aside....42
Table 12: RVFI signals and their kestrel aggregation....58
Table 13: The thirteen decode outputs....63
Table 14: Decode rows — OP, OP-IMM, LUI, AUIPC....69
Table 15: Decode rows — LOAD, STORE, BRANCH, JAL, JALR....70
Table 16: Decode rows — SYSTEM, MISC-MEM, and the illegal default....71
Table 17: The kestrel memory interface....73
Table 18: Halt causes and their spec counterparts....84
Table 19: rv32ui battery at func level — 42/42 with lockstep; every test halts on ecall (cause 1) with gp = 1, except ma_data, whose expected halt is the misaligned load (cause 4)....90
Table 20: Critical-path content per instruction class....111

List of Waveforms

No waveforms in this document.

1 Simplified Rv32I Index

Generated: 2026-10-23

2 Introduction

2.1 What this book is

This book is the condensed study-notes edition of RV32I as implemented by kestrel, the single-cycle core at rung 1 of the falcon suite. It is not a replacement for the RISC-V specification — the specification (`references/riscv-spec.pdf`, Volume I: Unprivileged Architecture) is the authority, and this book cites it by chapter and section throughout. What this book adds is the mapping from spec prose to silicon: which decode rows kestrel actually implements, what its memory contract pins down, how its retire interface reports execution, and how it was verified hard enough to trust.

Everything here describes the as-built core in `common-rv32i/rtl/includes/rv32i_pkg.sv`, `rtl/fub/kestrel_regfile.sv`, `common-rv32i/rtl/fub/rv32i_alu.sv`, `common-rv32i/rtl/fub/rv32i_imm_gen.sv`, `common-rv32i/rtl/fub/rv32i_decode.sv`, and `rtl/top/kestrel_core.sv`. Where a design decision was made that the spec leaves open — and RV32I leaves several doors open — the decision is stated plainly and labeled as a decision, with the spec's own permission cited.

2.2 What kestrel is

kestrel is a single-cycle RV32I core:

- One instruction retires per clock; the entire datapath completes in one cycle. (Behind block RAM a load takes a second clock while the memory holds the core; the core itself has no multi-cycle instruction.)
- Control is a combinational decode truth table, not a finite-state machine. There is no time dimension to control yet.
- The complete architectural state is the PC register, the 32-entry register file, and a handful of bookkeeping flops — three in total.
- There is no trap machinery. ECALL, EBREAK, illegal instructions, misaligned control-flow targets and misaligned loads/stores halt the core with a cause code on a dedicated port, and the halting instruction retires as an RVFI trap beat (see Chapters 3 and 5).
- Retirement is observed through a first-class RISC-V Formal Interface (RVFI) port bundle, so an external checker can diff the core against a golden model instruction by instruction.

The name follows the suite's convention: falcon species in strict size order, one per rung. The kestrel is the smallest falcon — the hovering falcon — and kestrel the core is the smallest machine in the suite: the whole of it is visible in one cycle.

2.3 Why single-cycle comes first

A single-cycle core is the honest starting point for the ladder because it removes every confound. When one instruction occupies the entire clock, there are no pipeline registers to reason about, no hazards, no forwarding priority, no speculation, no precise-exception problem. The register file either read the right operands or it didn't; the ALU either computed the right value or it didn't; the truth table either decoded the right controls or it didn't. Every verification failure points at exactly one combinational block.

That transparency is bought with a cost, and the cost is the subject of Chapter 7: the clock period must cover the slowest instruction's path through the whole machine, so the single-cycle discipline taxes every other instruction. Rung 2 (merlin) exists precisely because of that tax. But you cannot appreciate what a pipeline buys until you have traced one full cycle by hand, and you cannot debug a pipeline's hazard logic until you know what the correct answer was. kestrel is where the correct answer lives.

2.4 How to read this book

Chapters 2 through 5 are reference: the ISA, the datapath, the truth table, the memory contract. Chapter 6 is the verification record — read it even if you skip the RTL chapters, because it is also the story of what each verification method can and cannot see. Chapter 7 is the argument for the next rung.

RTL identifiers appear in code font exactly as they do in the sources (`ls_misalign`, `rd_wen_eff`, `HALT_IALIGN`), so statements in this book can be checked against the code mechanically.

Source: RISC-V Instruction Set Manual, Volume I, chapters 1-2 (specification edition riscv-spec.pdf); falcon-suite design spec `docs/superpowers/specs/2026-10-06-riscv-falcon-suite-design.md`

3 The Falcon Ladder

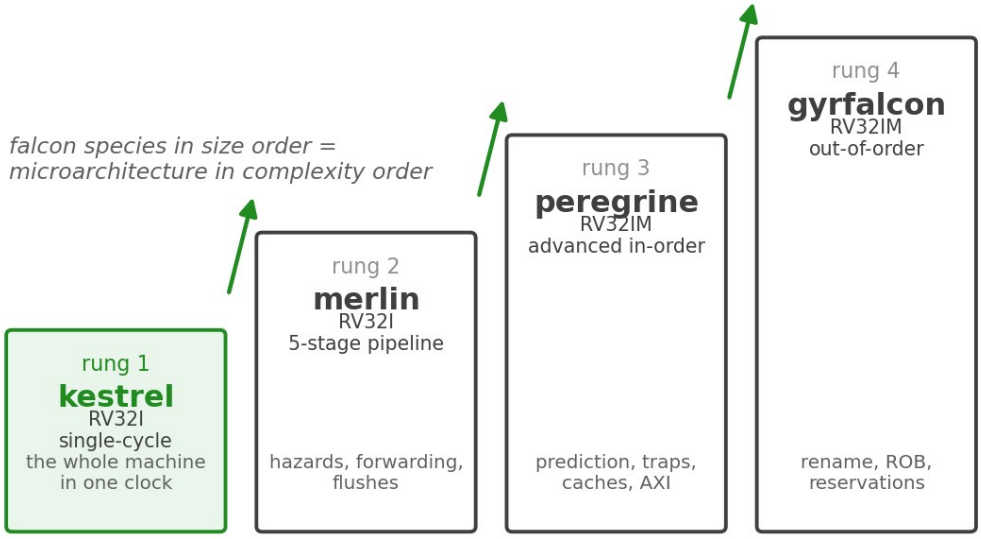
3.1 The suite

The falcon suite is a sequence of four RISC-V cores, one microarchitecture per rung, each rung a complete, verified, documented design small enough to hold in your head. Falcon species come in strict size order, and so do the cores: the smallest falcon is the simplest machine.

The falcon ladder

Rung	Core	ISA	Microarchitecture
1	kestrel-rv32i	RV32I	Single-cycle; the whole machine in one clock
2	merlin-rv32i	RV32I	Five-stage in-order pipeline (IF/ID/EX/MEM/WB)
3	peregrine-rv32im	RV32IM	Advanced in-order: branch prediction, traps, L1 caches, AXI master
4	gyrfalcon-rv32ima	RV32IMA	Out-of-order: rename, reorder buffer, reservation stations

3.1.1 Figure 1.1: The falcon ladder, smallest to largest



The falcon ladder: four rungs from single-cycle to out-of-order

The ladder is the point. Each rung exists to teach exactly one layer of computer-architecture machinery, and each rung's verification story reuses the rung below it. kestrel's decode truth table becomes merlin's per-stage control; merlin's forwarding network becomes the baseline peregrine speculates on top of; peregrine's precise traps become the commit contract gyrfalcon's reorder buffer enforces.

3.2 What each rung teaches

- **kestrel (this book).** Instruction formats, the datapath, control truth tables, the memory contract, and why single-cycle costs what it costs. Everything is visible in one cycle.
- **merlin.** Pipeline registers, RAW hazards, forwarding priority, the load-use bubble, and control-hazard flushes. The ISA is deliberately identical to kestrel's, so students diff behavior, not ISA, on the way up.
- **peregrine.** Speculation and the memory hierarchy: a branch predictor, real trap handling with precise state, and L1 caches behind an AXI master. The ISA grows to RV32IM.
- **gyrfalcon.** Out-of-order execution: register renaming, a reorder buffer, and reservation stations — the machine that hides program order.

3.3 Why kestrel is rung 1

The suite's design rule is that you meet each concept at the smallest machine that exhibits it. Single-cycle execution exhibits everything the ISA promises and nothing the microarchitecture adds — the ideal baseline. It is also the cheapest rung to verify exhaustively: a combinational core with an RVFI port drops directly into riscv-formal's instruction-check flow, and its traces diff cleanly against both a golden interpreter and the spike emulator.

Two consequences shape this book. First, kestrel documents its simplifications honestly — the CSR stubs, the FENCE NOPs, the halt instead of trap — because rung 3 is where those simplifications get replaced by real machinery, and the suite wants the seam visible. Second, kestrel keeps the ISA surface identical to merlin's so the rv32ui battery, the spike lockstep, and (adapted) the formal proofs all carry over. A rung-1 verification investment pays off at rung 2.

Source: falcon-suite design spec, "Naming and placement" and "Rung 1 — kestrel"; Patterson and Séquin, "RISC I: A Reduced Instruction Set VLSI Computer" (ISCA 1981) — the original see-everything machine (microarchitecture-papers.md, entry 1)

4 Conventions

4.1 Numbers and identifiers

- Hexadecimal values are written with a `0x` prefix or SystemVerilog style (`32'hFFFF_FFFE`); binary encodings use SystemVerilog sized literals (`7'b0110011`).
- RTL signal and parameter names appear in code font exactly as in the sources (`dmem_wstrb`, `RESET_ADDR`), so every claim can be grepped.
- Instruction names (ADD, JALR) are uppercase; assembly mnemonics in prose are lowercase (`add x3, x1, x2`).

4.2 Register names

RV32I defines thirty-two 32-bit registers, x0 through x31 (spec Volume I, section 2.1.1). This book uses the ABI names where they aid recognition — x1 is `ra` (return address), x2 is `sp`, x3 is `gp`, x10 and x11 are `a0`/`a1` — and the bare numbers elsewhere. One register is special: x0 always reads as zero, and writes to it are discarded. kestrel enforces both properties in the register file itself.

4.3 Halt versus trap

This book uses the two words the way the core uses its ports:

- A **trap**, per the spec (Volume I, section 1.6), is a control transfer to a handler. kestrel has no trap machinery and never traps.
- A **halt** is kestrel's stop condition: the core freezes the PC, suppresses writeback, and raises a cause code on `halt`/`halt_cause`. ECALL, EBREAK, illegal instructions, misaligned control-flow targets and misaligned loads/stores all halt. On RVFI the halting instruction still retires — as a beat with `rvfi_trap` set — so observers see the architecturally final state.

When the spec mandates a trap and kestrel substitutes a halt, the text says so explicitly, because that substitution is a documented design decision with a verification consequence (Chapter 6).

4.4 RVFI

RVFI (RISC-V Formal Interface) is the retire-port convention from the riscv-formal project: one record per retired instruction carrying PC, instruction word, register read/write, and memory access details. kestrel's RVFI aggregation rules are given in Chapter 3; the field-level semantics follow riscv-formal's `rvfi.md`.

4.5 Citations

The specification is cited by chapter and section (`Volume I, section 2.1.6`), never by page — the pinned PDF is a rolling intermediate release whose pagination can drift. Research literature is cited by entry in `references/microarchitecture-papers.md`, the suite's reading list, in the form (papers, entry N).

Source: RISC-V Instruction Set Manual, Volume I, sections 1.6 and 2.1.1; vendor/riscv-formal/docs/rvfi.md

5 Instruction Formats

5.1 Six formats, one word

Every RV32I instruction is exactly one 32-bit word (spec Volume I, sections 1.5 and 2.1.2). There is no compressed form at this rung: IALIGN=32, so instructions are always 4-byte aligned and the low two bits of the PC are always zero. That single fact is what makes kestrel's instruction-side misaligned policy (Chapter 5) so crisp.

Six formats cover the thirty-seven instructions. Register fields are 5 bits; the opcode is 7 bits at [6:0]; funct3 and funct7 qualify the operation within major opcodes.

The six RV32I instruction formats (spec Volume I, section 2.1.2)

Bits	31:25	24:20	19:15	14:12	11:7	6:0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]	rs1	funct3	rd	opcode	
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]	rd	opcode			
J	imm[20|10:1|11|19:12]	rd	opcode			

Formats with fewer than six fields pack the remaining bits into the immediate, which is why U and J rows above leave cells empty.

5.2 How kestrel decodes a format

kestrel's `rv32i_decode` keys its truth table on the triple `{opcode, funct3, funct7}` — the exact fields that distinguish every base instruction. `rv32i_imm_gen` does not wait for it: the format is a function of the opcode alone, so it builds all five immediates in parallel and picks one by `insn[6:2]` (decode's `imm_sel` output still exists but is unused). The full table is Chapter 4; the format layer is worth understanding on its own because the immediate encodings are the least obvious part of the ISA.

5.3 Immediate assembly

Only one of the six formats (I) has its immediate in one contiguous slice. The others scatter immediate bits around the register fields — an artifact of keeping register specifiers in fixed positions so the register file can read while the immediate is still assembling (spec Volume I, section 2.1.3, records this rationale). The built 32-bit signed immediate for each format:

Immediate assembly per format (spec Volume I, section 2.1.3)

Format	Immediate assembly	Used by
I	sext(insn[31:20])	ADDI, loads, JALR, shifts-by-immediate
S	sext({insn[31:25], insn[11:7]})	stores
B	sext({insn[31], insn[7], insn[30:25], insn[11:8], 1'b0})	branches
U	{insn[31:12], 12'b0}	LUI, AUIPC
J	sext({insn[31], insn[19:12], insn[20], insn[30:21], 1'b0})	JAL

`sext` is sign-extension from bit 31 of the assembled value. B and J immediates append an explicit 0 in bit 0 because branch and jump targets are always 2-byte-aligned at minimum — and with IALIGN=32, kestrel requires 4.

`rv32i_imm_gen` implements exactly this table: five assemblies with the sign bits replicated by concatenation, and a `unique case` on the major opcode `insn[6:2]` picking one. U-type needs no sign extension work beyond the shift; the spec's canonical form zero-extends the 20-bit field and places it in the upper bits, which is what the RTL does.

5.4 Shift immediates

RV32I encodes the shift amount for SLLI/SRLI/SRAI in the I-immediate's low 5 bits (`insn[24:20]`, shamt), with `insn[30]` selecting arithmetic versus logical right shift for the SR family and `insn[31:25]` required to be `0000000` (SLLI, SRLI) or `0100000` (SRAI). kestrel's decode keys on the full funct7 field, so any other high field is an illegal instruction rather than a shift — matching the spec's reserved-encoding rules (spec Volume I, section 2.1.4).

Source: RISC-V Instruction Set Manual, Volume I, sections 1.5, 2.1.2, 2.1.3, 2.1.4; `common-rv32i/rtl/fub/rv32i_imm_gen.sv`, `common-rv32i/rtl/fub/rv32i_decode.sv`

6 The Instruction Set

6.1 The thirty-seven

RV32I is thirty-seven instructions. kestrel implements all of them, plus documented behavior for the FENCE/FENCE.I and SYSTEM opcode classes that the base set surrounds itself with. The groups below follow the spec's own order (Volume I, section 2.1); the right-hand column is the spec section each group is defined in.

6.1.1 Integer register-register (R-type)

OP group: ten register-register instructions

Instruction	funct3/funct7	Operation	Spec
ADD	000 / 0000000	rd = rs1 + rs2	2.1.4.2
SUB	000 / 0100000	rd = rs1 - rs2	2.1.4.2
SLL	001 / 0000000	rd = rs1 << rs2[4:0]	2.1.4.2
SLT	010 / 0000000	rd = (rs1 <s rs2) ? 1 : 0	2.1.4.2
SLTU	011 / 0000000	rd = (rs1 <u rs2) ? 1 : 0	2.1.4.2
XOR	100 / 0000000	rd = rs1 ^ rs2	2.1.4.2
SRL	101 / 0000000	rd = rs1 >>u rs2[4:0]	2.1.4.2
SRA	101 / 0100000	rd = rs1 >>s rs2[4:0]	2.1.4.2
OR	110 / 0000000	rd = rs1 | rs2	2.1.4.2
AND	111 / 0000000	rd = rs1 & rs2	2.1.4.2

6.1.2 Integer register-immediate (I-type)

OP-IMM group: nine register-immediate instructions

Instruction	funct3	Operation	Spec
ADDI	000	rd = rs1 + sext(imm)	2.1.4.1
SLTI	010	rd = (rs1 <s	2.1.4.1

Instruction	funct3	Operation	Spec
		sext(imm)) ? 1 : 0	
SLTIU	011	rd = (rs1 <u sext(imm)) ? 1 : 0	2.1.4.1
XORI	100	rd = rs1 ^ sext(imm)	2.1.4.1
ORI	110	rd = rs1 | sext(imm)	2.1.4.1
ANDI	111	rd = rs1 & sext(imm)	2.1.4.1
SLLI	001, shamt	rd = rs1 << shamt	2.1.4.1
SRLI	101, shamt, insn[30]=0	rd = rs1 >>u shamt	2.1.4.1
SRAI	101, shamt, insn[30]=1	rd = rs1 >>s shamt	2.1.4.1

SLTIU compares against the sign-extended immediate interpreted unsigned — a famously surprising detail; `sltiu rd, rs1, -1` tests against 0xFFFFFFFF. kestrel implements it with the ALU's unsigned less-than on the same sign-extended immediate everyone else gets, so the surprise is preserved exactly.

6.1.3 Loads and stores

LOAD and STORE groups: five loads, three stores

Instruction	funct3	Size	Sign	Spec
LB	000	byte	signed	2.1.6
LH	001	halfword	signed	2.1.6
LW	010	word	n/a	2.1.6
LBU	100	byte	zero	2.1.6
LHU	101	halfword	zero	2.1.6
SB	000	byte	n/a	2.1.6
SH	001	halfword	n/a	2.1.6

Instruction	funct3	Size	Sign	Spec
SW	010	word	n/a	2.1.6

Effective address is rs1 + sext(imm) for both groups. Loads sign-extend (LB, LH) or zero-extend (LBU, LHU) into rd; stores source rs2. The memory side of these eleven instructions — alignment policy, byte strobes, the misaligned-access halt — is the subject of Chapter 5.

6.1.4 Branches

BRANCH group: six conditional branches

Instruction	funct3	Condition (taken when)	Spec
BEQ	000	rs1 == rs2	2.1.5.2
BNE	001	rs1 != rs2	2.1.5.2
BLT	100	rs1 <s rs2	2.1.5.2
BGE	101	rs1 >=s rs2	2.1.5.2
BLTU	110	rs1 <u rs2	2.1.5.2
BGEU	111	rs1 >=u rs2	2.1.5.2

Target is PC + sext(B-immediate), relative to the branch's own address. Not-taken cost is identical to taken cost in a single-cycle machine — one cycle either way — which is one of the few places kestrel is *easier* than the spec's mental model.

6.1.5 Jumps and upper immediates

JAL, JALR, LUI, AUIPC

Instruction	Format	Operation	Spec
JAL	J	rd = pc+4; pc = pc + sext(imm)	2.1.5.1
JALR	I	rd = pc+4; pc = (rs1 + sext(imm)) & ~1	2.1.5.1
LUI	U	rd = imm (upper 20 bits)	2.1.4
AUIPC	U	rd = pc + imm	2.1.4

JALR masks the target's bit 0 (`JALR_ALIGN_MASK = 32'hFFFF_FFFE` in the core); with IALIGN=32, a target whose bits [1:0] are not `00` raises the instruction-address-misaligned halt instead of executing (Chapter 5). AUIPC forms the first half of the standard `auipc`/`jalr` pair for position-independent long jumps; LUI never passes through the ALU in kestrel — the writeback mux takes the immediate directly.

6.2 Reserved encodings

Every encoding not listed above — branch funct3 2-3, load/store funct3 3 and 6-7, MISC-MEM funct3 2-7, SYSTEM funct3 100, SYSTEM imm12 outside ECALL/EBREAK/MRET, shift funct7 mismatches — is an illegal instruction in kestrel: decode falls to the default row and the core halts with cause `4'hF` (Chapter 4 gives the table; Chapter 5 explains the halt). HINT encodings (base instructions writing x0, spec Volume I, section 2.1.9) execute as their base instruction; the x0 write is discarded, so the architectural effect is the NOP the spec recommends.

Source: RISC-V Instruction Set Manual, Volume I, sections 2.1.4, 2.1.5, 2.1.6, 2.1.9; `common-rv32i/rtl/fub/rv32i_decode.sv`, `rtl/top/kestrel_core.sv`

7 The System Layer: Stubs, NOPs, and Halts

7.1 What RV32I surrounds itself with

The thirty-seven base instructions sit inside two opcode classes that belong to the wider system: MISC-MEM (FENCE and friends, spec Volume I, section 2.1.7, with FENCE.I split into the Zifencei extension, section 4.1) and SYSTEM (ECALL, EBREAK, and the CSR instructions of Zicsr, section 4.2, with MRET defined in the privileged Volume II). kestrel has no memory ordering hardware, no CSR file, and no trap handler, so these classes retire through documented stubs and NOPs rather than real machinery. This chapter states exactly what that means — including what it does not mean.

7.2 FENCE and FENCE.I retire as NOPs

kestrel's imem and dmem are separate, combinational-read ports on a single-cycle core: there is no write buffer to drain, no cache to invalidate, no instruction prefetch to refetch. Fence semantics are therefore vacuous at this rung, and decode retires FENCE (MISC-MEM funct3 000) and FENCE.I (funct3 001) as NOPs — one cycle each, no architectural effect. MISC-MEM funct3 2-7 are reserved and halt as illegal instructions.

One subtlety is worth pinning down because it surprises people: the *core* is Harvard, but the *testbench memory* is a single unified array that serves both fetch and load/store. That unification is a testbench choice, made so rv32ui-p-fence_i's self-modifying code works with no coherence path (see Chapter 6). It is not a core property, and the book does not pretend otherwise: kestrel the core never sees its own stores land in the instruction stream unless the memory system chooses to show them.

7.3 ECALL and EBREAK halt

ECALL and EBREAK exist to transfer control to an execution environment (spec Volume I, section 2.1.8). kestrel has no execution environment beyond halt-and-observe, so both halt the core: ECALL with cause `4'h1`, EBREAK with cause `4'h2`. The halting instruction retires as an RVFI trap beat. In practice ECALL is the suite's program-termination convention: every riscv-tests program ends in the p-env's `RVTEST_PASS`/`RVTEST_FAIL` sequence, whose final `ecall` is how kestrel knows the program is done (Chapter 6 reads the verdict off the `gp` register's last writeback).

7.4 MRET falls through

MRET (SYSTEM funct3 000, imm12 `0x302`) returns from a machine trap. It is meaningful only when traps exist — trap state, `mepc`, privilege transfers, all of which are rung-3 machinery (privileged Volume II, chapter 3). kestrel retires MRET as a documented NOP-with-fall-through: the PC advances to pc+4. That choice is not arbitrary: the riscv-tests p-environment points `mepc` at the instruction following its trap stub in the configurations kestrel runs, so pc+4 is exactly what the test expects. This is stated plainly so nobody mistakes it for trap support.

7.5 The CSR class retires as a bounded stub

The six Zicsr instruction forms — CSRRW, CSRRS, CSRRC and their immediate variants (SYSTEM funct3 001/010/011/101/110/111) — retire through a stub: `csr_stub` in decode raises `rd_wen`, and the writeback mux substitutes a hard zero. Reads return zero; writes drop on the floor. SYSTEM funct3 100 is reserved and halts as illegal.

Say it plainly: **this is not CSR support.** There is no CSR file, no `mstatus`, no `mtvec`, no counters, and nothing the instructions do has architectural effect beyond a zero written to rd. The stub exists for one bounded reason: the riscv-tests p-environment preamble executes about a dozen CSR instructions plus an MRET before every test body, and without the stub the battery cannot run at all. On the battery programs the only CSR read with rd != x0 is `mhartid` (value 0), which the stub matches. Anything needing real CSR state — trap handlers, counters, feature probing — is out of contract at this rung and fails loudly either as a zero where a nonzero was expected or as a program that halts where it should have trapped.

The honesty matters for the ladder: merlin inherits the same stub, and peregrine replaces it with a real machine-mode CSR file. The seam is deliberately visible in the decode table.

7.6 Halt causes, collected

The six halt causes (encoding centralized in `common-rv32i/rtl/includes/rv32i_pkg.sv`)

Cause	Name	Raised by	Spec would say
`4'h1`	HALT_ECALL	decode, ECALL	environment call
`4'h2`	HALT_EBREAK	decode, EBREAK	breakpoint
`4'h3`	HALT_IALIGN	core, misaligned taken control transfer	instruction-address-misaligned trap
`4'h4`	HALT_LALIGN	core, misaligned load	load-address-misaligned trap
`4'h5`	HALT_SALIGN	core, misaligned store	store-address-misaligned trap
`4'hF`	HALT_ILL	decode, default row	illegal instruction

Causes 1, 2, and F come from decode; causes 3-5 can only be raised by the core, because they need the resolved next PC and branch decision, or the access address, which decode cannot see. Chapter 5 develops causes 3-5 and the alignment rules behind them.

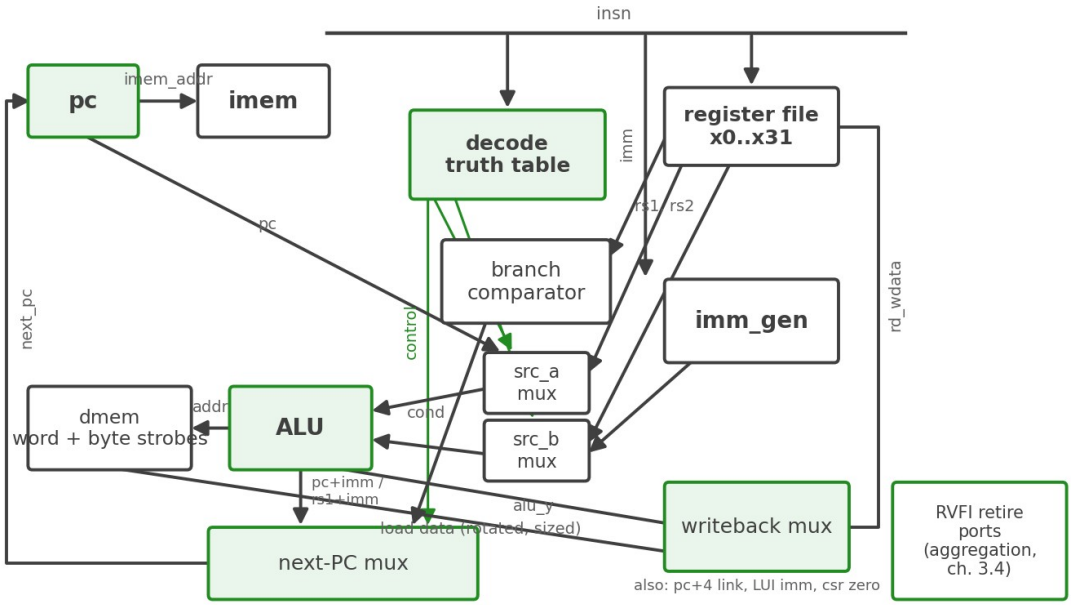
Source: RISC-V Instruction Set Manual, Volume I, sections 2.1.7, 2.1.8, 4.1, 4.2; Volume II, chapter 3 (MRET, machine trap state); `common-rv32i/rtl/fub/rv32i_decode.sv`, `rtl/top/kestrel_core.sv`, `common-rv32i/rtl/includes/rv32i_pkg.sv`

8 Datapath Overview

8.1 The machine in one picture

kestrel is six SystemVerilog modules and one package. Fetch is a wire: `imem_addr` is driven directly by the PC register and `imem_rdata` is the instruction. Everything else — decode, immediate generation, register read, execute, memory access, writeback, next-PC selection, retirement reporting — is combinational logic wrapped around a handful of state elements.

8.1.1 Figure 3.1: kestrel block diagram



kestrel datapath block diagram

The diagram is drawn to match the RTL: the blocks are the module instances in `kestrel_core.sv`, and every named wire is a real signal in the source.

8.2 The module list

kestrel's six modules

Module	Role	State
`rv32i_pkg`	Shared enums (`alu_op_e`, `imm_sel_e`) and the halt-cause encodings	none
`kestrel_regfile`	32 x 32-bit registers, two combinational read ports, one synchronous write, x0 tied to zero	32 words
`rv32i_imm_gen`	Builds the 32-bit immediate for the five formats	none
`rv32i_alu`	Ten operations: ADD SUB AND OR XOR SLL SRL SRA SLT SLTU	none
`rv32i_decode`	Combinational truth table from `{opcode, funct3, funct7}` to a thirteen-field control bundle	none
`kestrel_core`	Wires the leaves together: source muxes, branch comparator, next-PC mux, writeback mux, load/store rotation, halt, RVFI	5 flops + counters

8.3 The state inventory

Everything kestrel remembers, outside the register file:

The complete kestrel state inventory, register file aside

Element	Width	Purpose
`pc`	32	Program counter; resets to `RESET_ADDR` (default 0)
`retire_count`	64	RVFI retirement order: 0, 1, 2, …
`halt_q`	1	Halt holding register; latches any halt cause

That is the whole machine. Three flops plus a 32-word register file is the entire sequential content of a core that passes the rv32ui battery and the riscv-formal ISA proofs — the single-cycle discipline buys its simplicity honestly, and this table is the receipt. There is no FSM anywhere: the halt bit is a plain holding flop, and Chapter 4 shows why no more control state exists.

The register file itself has no reset (2026-10-20): it is plain storage, so it fits in distributed RAM rather than 1,024 flops. A register is undefined until a program writes it, and keeps an earlier program's value across a reload, so every program initializes each register it reads. The riscv-tests p-env does exactly that at its reset vector; the golden interpreter fails any test program that does not (Chapter 6). x0 is the one exception, and the core handles it by writing: entry 0 gets zero while the core is in reset and on every instruction that names x0 as its destination, so nothing on the read side has to special-case it.

8.4 Reading the chapters that follow

Chapter 3's remaining sections trace one cycle end to end (section 2), catalog the state and its gating (section 3), and specify the RVFI retire interface (section 4). Chapter 4 opens the decode truth table itself.

Source: `rtl/top/kestrel_core.sv`, `rtl/fub/kestrel_regfile.sv`, `common-rv32i/rtl/fub/rv32i_decode.sv`, `common-rv32i/rtl/fub/rv32i_alu.sv`, `common-rv32i/rtl/fub/rv32i_imm_gen.sv`, `common-rv32i/rtl/includes/rv32i_pkg.sv`

9 One Cycle, End to End

9.1 The clock in one sentence

On every rising edge, kestrel writes one register-file entry (maybe), updates the PC (unless held), and moves its bookkeeping flops — while a combinational cloud standing between `pc` and the next edge computes everything the next cycle needs: the fetched instruction, its decode, the operands, the result, the memory access, the next PC, and the RVFI record for whatever retires this cycle.

The sections below follow the data in dependency order, not file order.

9.2 Fetch and decode

`imem_addr = pc` and `insn = imem_rdata` — the instruction memory is expected combinationally: address in at the cycle, word out in the same cycle (the contract is Chapter 5). Decode is the truth table of Chapter 4, keyed on `{opcode, funct3, funct7}`, producing the thirteen-field control bundle (`alu_op`, `imm_sel`, `alu_src_a_pc`, `alu_src_b_imm`, `rd_wen`, `dmem_req`, `dmem_we`, `dmem_size`, `branch`, `jump`, `jalr`, `halt_cause`, `csr_stub`) plus defaults of "no operation, no halt."

9.3 Operands and immediates

The register file reads `rs1 = insn[19:15]` and `rs2 = insn[24:20]` combinationally; x0 reads zero because the file ties it. The immediate generator builds all five format immediates and picks one by the opcode alone, in parallel with the register read (decode's `imm_sel` is unused). Two muxes select the ALU inputs: `alu_src_a` is the PC (for AUIPC and branch targets) or `rs1_data`; `alu_src_b` is the immediate or `rs2_data`.

9.4 Execute

The ALU computes one of ten operations. Its internal `eq/lt/ltu` flags are deliberately left unconnected at the instance: with `src_a = PC` for branches those flags would compare pc against imm, not rs1 against rs2. Branch conditions come from a dedicated comparator in the core — `cmp_eq`, `cmp_lt` (signed), `cmp_ltu` (unsigned) on the register data — with funct3 selecting the condition in a `unique case`. In a single-cycle machine this parallel evaluation is free: the branch decision and the pc+imm target are both ready when the next PC is chosen, so taken and not-taken branches cost the same cycle.

9.5 Next-PC selection

The next-PC mux has exactly four rows:

1. JAL: `pc + imm` (J-immediate).
2. JALR: `(rs1_data + imm) & JALR_ALIGN_MASK` with `JALR_ALIGN_MASK = 32'hFFFF_FFFE` — bit 0 cleared, per spec Volume I, section 2.1.5.1.
3. Taken branch: `pc + imm` (B-immediate).
4. Fallthrough: `pc + 4`.

When taken control flow resolves to an address whose low two bits are not `00`, the core raises HALT_IALIGN instead of updating the PC — Chapter 5 covers the rule and why it lives in the core rather than decode.

9.6 Memory access

For loads and stores, `dmem_addr` is the word-aligned ALU result, `dmem_wdata`/`dmem_wstrb` are the store data and byte enables rotated into position, and read data is rotated back before size/sign selection implements LB/LBU/LH/LHU/LW. A naturally aligned access completes in the one cycle; a misaligned one halts and performs nothing. The full story — rotation math, the halt, RVFI encoding — is Chapter 5's.

9.7 Writeback

`rd_wdata` is selected by a five-row priority mux:

1. `csr_stub` — hard zero (the CSR stub, Chapter 2).
2. LUI — the U-immediate, straight from the generator, never through the ALU.
3. Load — the assembled, sign-extended read data.
4. JAL/JALR — `pc + 4`, the link value.
5. Everything else — `alu_y`.

The write enable is `rd_wen_eff = rd_wen & ~halt_q & ~mem_wait & ~misalign_link & ~ls_misalign` — equivalent to gating with `halt`: decode's `rd_wen` qualified by halt (the misaligned-control-flow halt retires JAL/JALR encodings, and the misaligned-load halt retires loads, whose decode `rd_wen` is set, so the gate is halt itself, not decode) and by the memory wait of a block-RAM load. The register file discards writes to x0 independently.

9.8 Two worked examples

`add x3, x1, x2`: decode selects ADD with both source muxes on registers; the ALU's `alu_y` passes the priority mux (row 5); x3's entry is written on the next rising edge; `next_pc = pc + 4`.

`bne x5, x6, -8` (a backward taken edge): the comparator's `~cmp_eq` is selected by funct3; the ALU simultaneously computes `pc + imm`; the next-PC mux row 3 wins; the PC register loads the target on the next edge and rvfi reports a beat with `pc_wdata` equal to that target.

Source: `rtl/top/kestrel_core.sv` (fetch, source muxes, ALU instance, comparator, next-PC mux, writeback mux); `common-rv32i/rtl/fub/rv32i_decode.sv`

10 Control State

10.1 Why there is no FSM

Classic processor control is a finite-state machine walking fetch, decode, execute, memory, writeback. kestrel doesn't have those phases: they are spatial, not temporal. Fetch is a wire off the PC register; decode is a truth table; execute is an ALU; memory access is a combinational port; writeback is a mux feeding a clocked register file. There is nothing to sequence, so there is no FSM — and no microcode, no control ROM, no phase counter.

The one place where a naive design would reach for an FSM — halting — is handled by a single holding flop with a plain set condition.

10.2 The halt holding register

`halt_q` is a one-bit flop that latches any nonzero `halt_cause_eff`:

```
halt_q <= halt_q | halt_now;
```

`halt_now` is the combinational OR of decode's causes (ecall, ebreak, illegal) and the core's own misaligned-target and misaligned-access conditions, masked while the memory holds the core (`mem_wait`). The external `halt` output is `halt_q | halt_now` so the first halting cycle is visible immediately. Once latched:

- The PC freezes (`pc <= (halt || mem_wait) ? pc : next_pc`).
- Writeback is suppressed (`rd_wen_eff` on the held halt and the halts an rd-writing instruction can raise, `rd_wb` on `halt`).
- Retirement stops: `rvfi_valid = rst_n & ~halt_q & ~mem_wait`, and since `halt_q` latches on the halting cycle, the halting instruction itself is the last beat — a trap beat with `rvfi_trap = halt_now`.
- The testbench watches halt-hold: after the trap beat, four more cycles of `halt` raised, `rvfi_valid` low, `rvfi_pc_rdata` frozen (Chapter 6).

A holding register rather than a pulse matters for verification: an external observer sampling any time after the halt sees a stable, unambiguous stopped state with the cause still on the port.

10.3 No retry bit, no merge register

Until 2026-10-20 kestrel completed misaligned loads and stores in hardware: a cross-word access took two memory beats, remembered by a retry flop (`ls_retry`) and a 32-bit merge register (`ls_rdata_lo`) holding the first word's bytes. On block RAM that retry logic sat on the critical path, so it was removed: a misaligned access now halts (Chapter 5), and those two flops left the inventory. No data is captured mid-instruction anywhere in kestrel.

10.4 The retirement counter

`retire_count` increments whenever `rvfi_valid` is high and backs `rvfi_order`, so retired instructions are numbered 0, 1, 2, … in program order forever. It is 64 bits wide — a 32-bit core will not wrap it — and it exists for exactly one consumer: riscv-formal's consistency checks, which use the order field to relate beats across cycles.

10.5 What this buys

Three flops with one-line next-state equations are reviewable in minutes and provable in bounded model checking with shallow depths (Chapter 6's formal checks run at depth 6 and cover reset plus memory wait states with headroom). The discipline the suite's style brief imposes — datapath logic is truth tables and muxes; FSMs only where control is unavoidable — shows up here as the reason kestrel's control can be exhaustively checked rather than merely simulated.

Source: `rtl/top/kestrel_core.sv` (halt register, PC register, retirement counter); falcon-suite RTL style brief

11 The RVFI Retire Interface

11.1 Retirement as the observation channel

kestrel was built RVFI-first: the retire port is not debug instrumentation bolted on afterward but the core's defined observation channel, present from the first vertical slice. Every instruction that retires produces one beat; the halting instruction produces one final trap beat; nothing else produces anything. A cocotb testbench diffs these beats field by field against a golden interpreter, and riscv-formal relates them to an instruction-set model (Chapter 6). Both consumers need the aggregation rules below to be exact.

11.2 The signal table

RVFI signals and their kestrel aggregation

Signal	Width	kestrel semantics
`rvfi_valid`	1	High exactly on cycles a beat retires: `rst_n & ~halt_q & ~mem_wait`
`rvfi_order`	64	`retire_count`; 0, 1, 2, … over retired beats
`rvfi_pc_rdata`	32	The retiring instruction's PC
`rvfi_pc_wdata`	32	`next_pc` — the architecturally next PC (the misaligned target on a cause-3 halt)
`rvfi_insn`	32	The instruction word
`rvfi_trap`	1	`halt_now`: high on the single halting trap beat
`rvfi_rs1_addr` / `rvfi_rs2_addr`	5+5	Register-file read ports (`insn[19:15]`, `insn[24:20]`)
`rvfi_rs1_rdata` / `rvfi_rs2_rdata`	32+32	Read data; x0 reads zero naturally
`rvfi_rd_addr` / `rvfi_rd_wdata`	5+32	Writeback, when `rd_wb`; both zero otherwise
`rvfi_mem_addr`	32	The word address of a load/store, zero otherwise (and on a misaligned, halting access)
`rvfi_mem_rmask` / `rvfi_mem_wmask`	4+4	The byte lanes the access covers within that word
`rvfi_mem_rdata` / `rvfi_mem_wdata`	32+32	The raw word read (loads) / size-truncated

Signal	Width	kestrel semantics
		store data shifted into its lanes, zero otherwise

11.3 The three gating rules

Three qualifications turn the raw datapath signals into a clean retire stream:

1. **x0 reports zero.** `rd_wb = rd_wen & (insn[11:7] != 0) & ~halt`, so a beat writing x0 reports `rd_addr = 0` with `rd_wdata = 0`. This is not merely tidiness: riscv-formal requires `rd_wdata == 0` whenever `rd_addr == 0`, and the early core failed exactly there until the rule was pinned. The register file independently discards the x0 write architecturally.
2. **A wait cycle is silent.** While a block-RAM memory holds the core with `mem_wait` (a load's first cycle), `rvfi_valid` is low: the instruction reports one beat, on the cycle it completes.
3. **Halt retires one trap beat, then silence.** `rvfi_trap = halt_now` marks the halting instruction itself (ecall, ebreak, illegal, misaligned target, or misaligned load/store) — a beat with trap set, no rd write, no memory fields — after which the latched `halt_q` holds `rvfi_valid` low forever. The wrapper used by riscv-formal derives `rvfi_halt = rvfi_valid & halt`, which matches the semantic exactly: the trap beat is the final retirement.

11.4 Memory-field packing

kestrel follows riscv-formal's `RISCV_FORMAL_ALIGNED_MEM` convention: `rvfi_mem_addr` is the word address and the masks mark the byte lanes the access covers inside it, the same lanes the store strobes drive. A load reports the raw word; a store reports its value truncated to the access size and shifted into its lanes, not the raw register — leaking the upper bytes into the payload is a real bug class this encoding prevents. (When kestrel still completed misaligned accesses in hardware it reported the unaligned byte address with masks packed from bit 0, riscv-formal's other convention.)

11.5 Why this pays

Because the rules are few and stated in RTL comments next to the assignments, the golden interpreter models the same rules in Python and the two can be diffed beat for beat over tens of thousands of instructions with zero tolerance. When a diff appears, it names the cycle, the PC, and the field — which is how the misaligned-jump defect of Chapter 6 was localized from a formal counterexample in minutes rather than debugged from waveforms.

Source: `rtl/top/kestrel_core.sv` (RVFI aggregation block); vendor/riscv-formal/docs/rvfi.md; `dv/tbclasses/kestrel/rv32i_interpreter.py`

12 The Control Bundle

12.1 Thirteen wires out of decode

`rv32i_decode` is a pure function from the 32-bit instruction word to a control bundle. It holds no state and no clock; the same word always produces the same bundle. The bundle's fields:

The thirteen decode outputs

Field	Type	Meaning when set / value
`alu_op`	enum	ALU operation: ADD SUB AND OR XOR SLL SRL SRA SLT SLTU
`imm_sel`	enum	Immediate format to build: I S B U J
`alu_src_a_pc`	bit	ALU left input is the PC (else rs1)
`alu_src_b_imm`	bit	ALU right input is the immediate (else rs2)
`rd_wen`	bit	The instruction writes rd this cycle
`dmem_req`	bit	A data-memory access is in flight
`dmem_we`	bit	The access is a store (else a load)
`dmem_size`	2 bits	Access size: 00 byte, 01 halfword, 10 word
`branch`	bit	Instruction is a conditional branch
`jump`	bit	Instruction is JAL
`jalr`	bit	Instruction is JALR
`halt_cause`	4 bits	Halt encoding (Chapter 2), `HALT_NONE` when running
`csr_stub`	bit	SYSTEM CSR class:

Field	Type	Meaning when set / value
		writeback a hard zero

12.2 The key: opcode, funct3, funct7

The truth table is a single `unique casez` keyed on `{opcode, funct3, funct7}` — a 17-bit key. Every RV32I instruction is distinguished by those three fields alone, which is why the table needs no other instruction bits at the top level. (The SYSTEM privileged row then re-cases on `insn[31:20]`, the imm12, to split ecall/ebreak/mret from the illegal encodings.)

Fields that carry immediate bits in some formats are keyed as don't-cares: `F7_DC = 7'b???????` matches OP-IMM arithmetic and every load, store, and branch row, since funct7 is immediate bits there. The shift rows are the exception — SLLI/SRLI/SRAI key funct7 fully, which is what makes mis-encoded shifts illegal rather than silently executed.

12.3 The default bundle

Before the case, decode assigns a default bundle: `alu_op = ADD`, `imm_sel = I`, every control bit clear, `halt_cause = HALT_NONE`. Each table row then sets only what it needs — most rows touch three or four fields. The `default:` row of the case changes exactly one field, `halt_cause = HALT_ILL`, which is the entire illegal-instruction mechanism. Keeping the default explicit means every encoding not claimed by a row halts, loudly, instead of executing as something plausible.

12.4 Reading the table

The next section gives the table in full, transcribed from `common-rv32i/rtl/fub/rv32i_decode.sv`. Notation is defined at the top of that section: the 17-bit key as `opcode.funct3.funct7`, `any` for don't-care key fields, abbreviated bundle columns (`alu`, `imm`, `aPC`, `bImm`, `rd`), non-default values only (– means the default), and `dmem` sizes in bytes (B/H/W).

Source: `common-rv32i/rtl/fub/rv32i_decode.sv`, `common-rv32i/rtl/includes/rv32i_pkg.sv`

13 The Decode Table

The full `unique casez ({opcode, funct3, funct7})` of `common-rv32i/rtl/fub/rv32i_decode.sv`, transcribed row for row. The key column is the 17-bit casez key written as `opcode.funct3.funct7`; `any` marks a don't-care key field. Bundle columns show only non-default values and are abbreviated to keep the table readable — `alu` is `alu_op`, `imm` is `imm_sel`, `aPC` is `alu_src_a_pc`, `bImm` is `alu_src_b_imm`, `rd` is `rd_wen`, and `dmem` reports `req`, `+we`, and the size in bytes (B/H/W). – means the default value.

13.1 Arithmetic and upper-immediate rows

The ALU computes datapath arithmetic here; for LUI the writeback mux bypasses the ALU and takes the immediate directly (Chapter 3).

Decode rows — OP, OP-IMM, LUI, AUIPC

opcode.funct3.funct7 (17-bit casez key)	Row	alu	imm	aPC	bImm	rd
0110011.000.0000000	ADD	ADD	—	—	—	1
0110011.000.0100000	SUB	SUB	—	—	—	1
0110011.001.0000000	SLL	SLL	—	—	—	1
0110011.010.0000000	SLT	SLT	—	—	—	1
0110011.011.0000000	SLTU	SLTU	—	—	—	1
0110011.100.0000000	XOR	XOR	—	—	—	1
0110011.101.0000000	SRL	SRL	—	—	—	1
0110011.101.0100000	SRA	SRA	—	—	—	1
0110011.110.0000000	OR	OR	—	—	—	1
0110011.111.0000000	AND	AND	—	—	—	1
0010011.000.any	ADDI	ADD	I	—	1	1
0010011.010.any	SLTI	SLT	I	—	1	1
0010011.011.any	SLTIU	SLTU	I	—	1	1
0010011.100.any	XORI	XOR	I	—	1	1
0010011.110.any	ORI	OR	I	—	1	1
0010011.111.any	ANDI	AND	I	—	1	1
0010011.001.0000000	SLLI	SLL	I	—	1	1
0010011.101.0000000	SRLI	SRL	I	—	1	1
0010011.101.0100000	SRAI	SRA	I	—	1	1
0110111.any.any	LUI	ADD	U	—	1	1
0010111.any.any	AUIPC	ADD	U	1	1	1

13.2 Loads, stores, branches, jumps

Every branch/jump row uses `alu = ADD` so the ALU produces `pc + imm` for the next-PC mux while the dedicated comparator decides taken-ness in parallel. JALR is the one jump whose ALU left input is rs1, not PC.

Decode rows — LOAD, STORE, BRANCH, JAL, JALR

opcode.funct3.funct7 (17-bit casez key)	Row	imm	aPC	bImm	dmem	ctl
0000011.000.any	LB	I	—	1	req, B	—
0000011.001.any	LH	I	—	1	req, H	—
0000011.010.any	LW	I	—	1	req, W	—
0000011.100.any	LBU	I	—	1	req, B	—
0000011.101.any	LHU	I	—	1	req, H	—
0100011.000.any	SB	S	—	1	req+we, B	—
0100011.001.any	SH	S	—	1	req+we, H	—
0100011.010.any	SW	S	—	1	req+we, W	—
1100011.000.any	BEQ	B	1	1	—	branch
1100011.001.any	BNE	B	1	1	—	branch
1100011.100.any	BLT	B	1	1	—	branch
1100011.101.any	BGE	B	1	1	—	branch
1100011.110.any	BLTU	B	1	1	—	branch
1100011.111.any	BGEU	B	1	1	—	branch
1101111.any.any	JAL	J	1	1	—	jump
1100111.000.any	JALR	I	—	1	—	jalr

13.3 System, fence, and the default

Decode rows — SYSTEM, MISC-MEM, and the illegal default

opcode	funct3	imm 12	Row	Behavior (how the row retires)	`halt_cause`
1110011	000	000	ECALL	halt; RVFI trap beat	`4'h1`
1110011	000	001	EBREAK	halt; RVFI trap beat	`4'h2`
1110011	000	302	MRET	retire as NOP, pc+4 fall-through	—
1110011	000	other	—	illegal	`4'hF`
1110011	001/010/011/101/110/111	any	CSRRW/S/C + imm forms	`csr_stub`: zero readback, writes dropped	—
1110011	100	any	—	reserved, illegal	`4'hF`
0001111	000	any	FENCE	retire as NOP	—
0001111	001	any	FENCE.I	retire as NOP	—
0001111	010-111	any	—	reserved, illegal	`4'hF`
any unclaimed	—	—	default	illegal instruction	`4'hF`

13.4 What the table teaches

Three design rulings visible in the table deserve explicit mention:

1. **Branch funct3 010/011, load/store funct3 011 and 110/111, MISC-MEM funct3 2-7, SYSTEM funct3 100, shift funct7 mismatches: all illegal.** RV32I leaves these encodings reserved (spec Volume I, sections 2.1.4 and 2.1.6); kestrel halts on them rather than guessing. A reserved encoding today may become an instruction tomorrow, and silently executing it is how compatibility bugs are born.
2. **The CSR class is one stub row-set, not six special cases.** All six Zicsr forms share `csr_stub = 1; rd_wen = 1` and nothing else; the writeback mux owns the zero. The table stays small because the stub is honest about there being nothing to decode.
3. **Cause 3 appears nowhere in this table.** HALT_IALIGN is raised only in `kestrel_core`, which can see the resolved next PC and the branch decision. Decode provably cannot know — a branch's target alignment depends on the register operands it never evaluates. Chapter 5 gives the condition.

Source: `common-rv32i/rtl/fub/rv32i_decode.sv` (the `unique casez` in full), `common-rv32i/rtl/includes/rv32i_pkg.sv` (HALT_* encodings), `rtl/top/kestrel_core.sv` (misaligned-target halt)

14 The Memory Contract

14.1 The interface

kestrel talks to memory through two ports that are deliberately minimal. There is no bus fabric, no wait states, no byte enables on the instruction side, and no burst machinery — at this rung the memory system is expected to keep up with the core, and the contract says so out loud.

The kestrel memory interface

Port	Direction	Width	Contract
`imem_addr`	out	32	Instruction fetch address, driven by the PC
`imem_rdata`	in	32	Combinational read data: the word at `imem_addr`, valid in the same cycle
`dmem_req`	out	1	A load or store is accessing data memory this cycle
`dmem_we`	out	1	The access is a store
`dmem_addr`	out	32	Word-aligned byte address (`{alu_y[31:2], 2'b00}`)
`dmem_wstrb`	out	4	Per-byte write enables, rotated into position for stores
`dmem_wdata`	out	32	Store data, rotated into position
`dmem_rdata`	in	32	Combinational read data for

Port	Direction	Width	Contract
			loads

Three properties define the contract:

1. **Harvard at the core.** Instruction fetch and data access are separate ports backed by whatever the integrator provides. The core never issues a data access to the instruction memory or vice versa, and it never sees coherence between them.
2. **Combinational reads, or an explicit wait.** Both memories answer in the same cycle the address is presented. This is the load-bearing simplification of the single-cycle design: an instruction's full path — fetch, decode, register read, ALU, data read, writeback — must fit in one clock. A synchronous-read memory (block RAM) may instead hold the core with `mem_wait`, during which nothing in the core changes. `kestrel_mem_bram` does that for a load's data read only: it fetches one clock early at the core's `imem_addr_next` (the PC after the coming edge), so the RAM's output register acts as the instruction register and every other instruction still takes one clock.
3. **Word-wide buses with byte strobes.** Data memory is 32 bits wide; sub-word stores rotate the data and strobe into the addressed byte lanes, and sub-word loads rotate read data back before size/sign selection. The testbench memory honors `dmem_wstrb` per byte, which is the contract that pins the rotation convention (Chapter 6).

14.2 Timing: one cycle, except when documented

An instruction that touches memory completes in exactly one clock; a misaligned access halts instead (next section of this chapter). There are no other exceptions beyond the memory's own `mem_wait`: no arbitration stalls, no refresh interference. The PC holds only for halts and while `mem_wait` is high. Everything else advances every cycle.

`RESET_ADDR` (default `32'h0000_0000`, a module parameter) selects the fetch address out of reset; the testbench also parameterizes it to prove the core starts anywhere.

14.3 What the contract deliberately excludes

- **No instruction-side data access.** Self-modifying code is invisible to the core unless the *memory system* unifies the ports. The kestrel testbench does exactly that — one array serving both fetch and load/store — as a testbench choice so rv32ui-p-fence_i runs without a coherence path. That unification is documented here so no integrator mistakes it for a core feature.
- **No misaligned instruction fetch.** IALIGN=32: fetch addresses are always 4-byte aligned, and the low PC bits are guaranteed by the next-PC logic plus the halt of section 3 of this chapter.
- **No memory protection, ordering, or atomicity.** RVWMO (spec Volume I, chapter 3) is satisfied vacuously by a core that issues at most one access at a time and never reorders; there is nothing to model.

Source: `rtl/top/kestrel_core.sv` (port list, L/S datapath); `dv/tb/kestrel_tb_top.sv` (unified testbench memory, wstrb-honoring store port); falcon-suite RTL style brief

15 Misaligned Data Accesses

15.1 What the spec permits

RV32I does not require misaligned load/store support, but it explicitly permits it: an execution environment may handle misaligned accesses in hardware, invisibly trap and emulate them, or trap and refuse, and it may treat accesses to side-effecting regions differently (spec Volume I, section 2.1.6). Naturally aligned accesses are guaranteed atomic; misaligned accesses are explicitly *not* required to be atomic. The design space is open, and kestrel takes the plainest corner of it: **a misaligned load or store halts the core.** Trap-and-refuse is the spec's own option; kestrel has no trap machinery, so, like every other trap, it halts (cause 4 for a load, 5 for a store).

This is a *data* policy. Instruction-side misalignment is never handled in hardware by anything, and section 3 of this chapter covers that halt.

15.2 The rotation model

Naturally aligned sub-word accesses are implemented by byte-lane rotation. For an access at byte offset `o` with size bytes `s`:

- **Offset:** `o = rs1[1:0] + imm[1:0]`, the address's low bits from their own two-bit add, so the rotation and the halt test do not wait for the 32-bit address.
- **Store:** `dmem_wdata = rs2_data << (8*o)` — the value slides into the addressed lanes; `dmem_wstrb = size_mask << o` — the byte enables slide with it. The memory merges enabled bytes into the word.
- **Load:** `rdata_shifted = dmem_rdata >> (8*o)` — the addressed bytes slide down to bit 0, and a final case implements LB (sign-extend from bit 7), LBU/LHU (zero-extend), LH, LW.

Bytes are always aligned. Halves are aligned at offsets 0 and 2, words at 0. Every aligned access fits inside one word, so the rotation is the entire mechanism and the access completes in the single cycle.

15.3 The misaligned halt

A half at an odd offset, or a word at any nonzero offset, raises `ls_misalign`. The access performs nothing: the store strobes are forced to zero, the load does not write rd, and the instruction retires as the usual RVFI trap beat with empty memory fields, then the core holds (section 3). rv32ui-p-ma_data, which needs hardware support, halts on its first misaligned load; spike, which traps misaligned accesses too, takes `trap_load_address_misaligned` at the same instruction, so the lockstep still lines up (Chapter 6).

15.4 Why not hardware?

Until 2026-10-20 kestrel did complete misaligned accesses in hardware: in-word ones by rotation, cross-word ones in two beats, with a retry flop, a 32-bit merge register for the first word's bytes, and the PC held for the extra cycle. On block RAM that logic sat on the critical path — it gated the register write and the PC hold, between the address add and the register file — and it was most of kestrel's control state. Dropping it leaves a rotation and one compare, which is the smaller and simpler design a rung-1 core should be. A program that needs misaligned access can split it into aligned byte or half accesses, as compilers already do.

Source: RISC-V Instruction Set Manual, Volume I, section 2.1.6; `rtl/top/kestrel_core.sv` (L/S rotation and the misaligned halt); `dv/tests/programs/ls_misaligned.s`, `ls_misalign_store.s`, `ls_chain.s`

16 IALIGN and the Misaligned-Target Halt

16.1 The rule

With IALIGN=32 — no compressed instructions — every instruction fetch must be 4-byte aligned. The spec (Volume I, section 2.1.5) therefore requires an *instruction-address-misaligned* exception when taken control flow — a taken branch, JAL, or JALR — targets an address whose low two bits are not `00`. RISC-V's permission to handle *data* misalignment in hardware does not extend here (kestrel halts on misaligned data anyway, section 2): executing at a misaligned instruction address is never an option.

16.2 Why decode cannot catch it

The condition needs the *resolved* next PC: for JALR it depends on a register value, and for every branch it depends on the taken decision. Decode sees neither — it produces the control bundle, not the data. kestrel's `rv32i_pkg` comments state the ownership directly: causes 1, 2, and F belong to decode; causes 3-5 belong to the core. In `kestrel_core.sv`:

```
misalign_target = (jump | jalr | branch_taken) & (next_pc[1:0] !=
2'b00);
```

— OR-ed into the halt path as `HALT_IALIGN` (`4'h3`), through the same holding register as every other halt: trap beat, writeback suppressed, PC frozen, cause on the port.

16.3 The writeback subtlety

JAL and JALR set decode's `rd_wen` — they are link instructions, and a taken JAL/JALR that *does* align must write `pc+4`. The misaligned case must not. Rather than teaching decode about alignment, the core gates both the register write and the RVFI report with `halt` itself: `rd_wen_eff = rd_wen & ~halt_q & ~mem_wait & ~misalign_link & ~ls_misalign` (the halts an rd-writing JAL/JALR or load can raise; equal to gating with `halt`, without the branch comparator on the path) and `rd_wb = rd_wen & (insn[11:7] != 0) & ~halt`. The trap beat reports the misaligned target on `rvfi_pc_wdata` (it is `next_pc`, and `next_pc` is the misaligned address), which is exactly what a real trap's state would record as the faulting target — a detail the formal model checks explicitly.

16.4 The six halt causes, in context

Halt causes and their spec counterparts

Cause	Encoding	Raised in	Spec's name for the event
ECALL	`4'h1`	decode	environment-call exception
EBREAK	`4'h2`	decode	breakpoint exception
misaligned control-flow target	`4'h3`	core	instruction-address-misaligned exception
misaligned load	`4'h4`	core	load-address-misaligned exception
misaligned store	`4'h5`	core	store/AMO-address-misaligned exception
illegal encoding	`4'hF`	decode	illegal-instruction exception

Every row is a case where RV32I mandates a trap and kestrel — having no trap machinery — substitutes a halt with the cause on a port. The RVFI trap beat keeps the event architecturally visible: riscv-formal's checks require exactly this reporting, and the suite's later rungs replace the halt with real trap delivery. This table is the seam.

16.5 The fence_i connection

rv32ui-p-fence_i is the one battery test whose program modifies its own instruction stream between fetches. The core's Harvard contract says nothing about store-to-fetch visibility, so the *testbench* provides it: one unified memory array serves `imem_rdata` and `dmem_rdata`, and FENCE.I's NOP retirement is sufficient because there is no cache to invalidate and no prefetch to refetch. The test passing is evidence of the testbench's coherence choice, not of a core feature — and the book says so because the distinction matters the moment a real cache enters at rung 3.

Source: RISC-V Instruction Set Manual, Volume I, sections 1.5, 2.1.5; `common-rv32i/rtl/includes/rv32i_pkg.sv` (cause encoding and ownership comment), `rtl/top/kestrel_core.sv` (misalign_target, writeback gating), `common-rv32i/rtl/fub/rv32i_decode.sv`; task-9 report (the formal CEX that pinned this)

17 Testbench and Golden Model

17.1 The harness

kestrel's testbench is deliberately thin — the core is the design under test and RVFI is the only observation channel it needs. `kestrel_tb_top` wraps the core in a behavioral 64 KiB word memory with combinational reads, loaded at time 0 from `+imem=<hex>` (and `+dmem=<hex>`) plusargs via `$readmemh`. The memory is *unified*: one array serves fetch and load/store, the testbench choice that lets rv32ui-p-fence_i's self-modifying code work with no coherence path (Chapter 5). The store port honors `dmem_wstrb` per byte — the contract that pins the rotated-strobe convention.

Two additions serve the battery (Chapter 6's next sections): a backdoor write port (`tb_mem_we`/`tb_mem_addr`/`tb_mem_wdata`, one word per cycle, deliberately not reset-gated) so all 42 vendor images stream through one Verilator build, and a tohost watch on the store port.

The cocotb side (`kestrel_tb.py`) owns the clock/reset choreography and the RVFI capture, and it enforces a sampling contract that took one real bug to learn: reset is released *right after a posedge* and beats are sampled on *falling edges*. Post-edge release plus negedge sampling captures every retiring instruction exactly once — including the first `RESET_ADDR` beat, which lives only in the partial cycle before the first posedge and was entirely missed by the naive post-edge sampling. The halting cycle is sampled too: the trap beat (`rvfi_valid=1`, `rvfi_trap=1`) is recorded, after which `rvfi_valid` stays low.

After halt, the TB samples four more cycles pinning halt-hold: `halt` stays raised, `rvfi_valid` stays low, `rvfi_pc_rdata` frozen.

17.2 The golden interpreter

`rv32i_interpreter.py` is a 476-line Python model of RV32I that emits the same RVFI beat format as the core — one record per retired instruction, none for anything that doesn't retire, plus the halt state. Its memory model mirrors the core's conventions exactly: word-based, rotated-strobe stores, the cause-4/5 halt on a misaligned load/store, the CSR stub's zero writeback, MRET fall-through, FENCE NOPs, and the cause-3 halt with the misaligned target on `pc_wdata`. When the RTL and the model disagree, the diff is a named field at a named beat at a named PC — minutes to localize, not a waveform archeology session.

The diff is total: every sampled beat is compared field by field across the entire RVFI bundle — pc, insn, order, trap, rs addresses and data, rd address and data, and all four memory channels. There is no sampled subset and no tolerance.

17.3 Programs and the hex pipeline

Hand-written assembly programs (`focus`, `rv32ui_ops`, branch/jump vectors, load/store vectors, system tests) are built by a small script:

```
as -> objdump (provenance) -> objcopy -O verilog -> normalize_hex.py -> <name>.hex
```

`objcopy -O verilog` emits *bytes* with byte-addressed `@` records, which the testbench's *word* arrays cannot load directly (an `@80000000` record would index out of bounds), so `normalize_hex.py` packs little-endian words and emits sparse word-indexed records, with a `--base` option that subtracts the battery images' `0x80000000` link address. Vendor riscv-tests hexes go through the same normalizer.

17.4 The run grammar

Every test runs at two Verilator build levels through the rds-dv framework, sources always from the filelists:

```
make -C projects/components/riscv-ip/kestrel-rv32i/dv/tests run-
<testroot>-<gate|func>[-serial]
```

`gate` is the fast smoke level; `func` runs the full matrix. The battery, lockstep, and formal layers of the next sections all sit on top of this harness and reuse its contracts rather than re-deriving them.

Source: `dv/tb/kestrel_tb_top.sv`, `dv/tbclasses/kestrel/kestrel_tb.py`, `dv/tbclasses/kestrel/rv32i_interpreter.py`, `dv/tests/programs/build_progs.sh`, `dv/tests/programs/normalize_hex.py`; task-5 report (sampling contract, fail-first evidence)

18 The rv32ui Battery

18.1 What the battery is

The `rv32ui-p-*` programs from the riscv-tests suite are the classic RISC-V architectural conformance battery: 42 self-checking RV32I tests, each linking the same p-environment preamble and ending in `RVTEST_PASS`/`RVTEST_FAIL`. kestrel runs all 42 at `func` level in a single Verilator build — the runner asserts reset, backdoor-loads the normalized image, releases reset, runs to halt, and scores — and three of them (simple, add, addi) again at `gate` as a smoke level. The summary table is the record (beats = RVFI retire beats; records = spike commit-log records, lockstep-checked as section 3 describes):

rv32ui battery at func level — 42/42 with lockstep; every test halts on ecall (cause 1) with gp = 1, except ma_data, whose expected halt is the misaligned load (cause 4)

Test (rv32ui-p-*)	Beats (RVFI retire)	Spike records (commit log)	Result (battery verdict)
add	500	5507	PASS / lockstep ok
addi	277	5284	PASS / lockstep ok
and	520	5527	PASS / lockstep ok
andi	233	5240	PASS / lockstep ok
auipc	93	5100	PASS / lockstep ok
beq	326	5333	PASS / lockstep ok
bge	344	5351	PASS / lockstep ok
bgeu	369	5376	PASS / lockstep ok
blt	326	5333	PASS / lockstep ok
bltu	351	5358	PASS / lockstep ok
bne	326	5333	PASS / lockstep ok
fence_i	331	5338	PASS / lockstep ok
jal	90	5097	PASS / lockstep ok
jalr	150	5157	PASS / lockstep ok
lb	288	5295	PASS / lockstep ok
lbu	288	5295	PASS / lockstep ok
ld_st	998	6005	PASS / lockstep ok
lh	304	5311	PASS / lockstep ok

Test (rv32ui-p-*)	Beats (RVFI retire)	Spike records (commit log)	Result (battery verdict)
lhu	313	5320	PASS / lockstep ok
lui	100	5107	PASS / lockstep ok
lw	318	5325	PASS / lockstep ok
ma_data	76	5083	PASS (expected halt: cause 4 at the first misaligned load) / lockstep ok
or	523	5530	PASS / lockstep ok
ori	240	5247	PASS / lockstep ok
sb	489	5496	PASS / lockstep ok
sh	542	5549	PASS / lockstep ok
simple	76	5083	PASS / lockstep ok
sll	528	5535	PASS / lockstep ok
slli	276	5283	PASS / lockstep ok
slt	494	5501	PASS / lockstep ok
slti	272	5279	PASS / lockstep ok
sltiu	272	5279	PASS / lockstep ok
sltu	494	5501	PASS / lockstep ok
sra	547	5554	PASS / lockstep ok
srai	291	5298	PASS / lockstep ok
srl	541	5548	PASS / lockstep ok
srli	285	5292	PASS / lockstep ok
st_ld	518	5525	PASS / lockstep ok
sub	492	5499	PASS / lockstep ok
sw	549	5556	PASS / lockstep ok
xor	522	5529	PASS / lockstep ok
xori	242	5249	PASS / lockstep ok

Gate level reruns simple, add, and addi — 3/3 pass, lockstep not applicable at that level by design.

18.2 The verdict protocol

The p-environment reports through a `tohost` mailbox: `RVTEST_PASS` stores 1 to tohost (via the trap-vector path, since a real core traps the terminating ecall), and the fesvr front end converts a tohost write into the emulator's exit code. kestrel works differently, and the difference is deliberate:

- The core halts *on* the terminating ecall (cause 1) by design, before any trap-vector `sw gp, tohost` can execute. The DUT never writes tohost — so the store port is watched for one anyway (the run contract is run-to-halt, tohost-write, or timeout, whichever first).
- The verdict is `gp`'s last RVFI writeback — x3 is exactly the register the p-environment would have stored, so its final value is the same information: 1 means pass, an odd value greater than 1 is a fail code.
- tohost is discovered per image, not hardcoded: a pure-Python ELF32 symbol reader finds it at `0x80001000` in 41 images and `0x80002000` in `ld_st` (its larger data section pushes `.tohost` past the usual address); `fromhost` follows at tohost + 0x40.
- Spike's exit code independently confirms tohost == 1 for every test on the lockstep side (next section), so the battery's verdict does not rest on its own convention alone.

18.3 What the battery proves — and its seam

Forty-two self-checking programs with a second emulator agreeing on every retired instruction is strong evidence of ISA conformance. Its seam is the p-environment preamble: every test runs the same ~12 CSR instructions and an MRET before its body, which is why the bounded CSR stub of Chapter 2 exists at all. And as Chapter 6's final section records, forty-two directed tests still missed a real core bug that formal methods found in an afternoon — the battery tells you the programs you thought of pass; it says nothing about the ones you didn't.

Source: task-8 report (battery table, tohost map, verdict protocol); `dv/tbclasses/kestrel/rv32ui_battery.py`, `dv/tests/test_kestrel_rv32ui.py`; `logs/rv32ui_battery/rv32ui_battery_summary_func.txt`

19 Spike Lockstep

19.1 What lockstep adds

The golden interpreter checks kestrel against kestrel's own understanding of RV32I — same author, same conventions, same blind spots. Spike, the RISC-V emulator, is an independent implementation: different code base, different authors, years of field use. Lockstepping the two means an disagreement is almost certainly kestrel's bug, and the suite pins spike 1.1.0 for reproducibility. Each battery test runs spike on the same ELF and the runner walks the two retirement streams together.

19.2 The mechanics

Spike is invoked as `spike --isa=RV32I -l --log=<file> <elf>` — the `-l` must accompany `--log`; `--log` alone redirects a disabled log and produces an empty file. This pinned build's commit line carries pc and instruction encoding only:

```
core   0: 0x0000000080000000 (0x00000297) auipc   t0, 0x0
```

There are no register-write or memory-access payloads in this build (checked against the source: `processor_t::disasm` prints pc, insn, and disassembly only). Consequently the lockstep is layered: the golden interpreter's full-field beat diff owns rd/rs/mem correctness, and spike owns the (pc, insn) retirement stream, the exception records, and the exit code — an independent check that the right instructions executed in the right order. Four format details are pinned by investigation:

1. **Boot prefix.** fesvr's boot ROM runs 5 instructions at 0x1000 before jumping to the ELF entry; the diff skips records until the first commit at `e_entry`.
2. **`Executed N times` lines.** Spike collapses back-to-back identical (pc, insn) commits; the parser expands them before the walk.
3. **Exception records.** `core   0: exception <kind>, epc 0x<pc>` (plus a `tval` line) interleave with commits; the trapping instruction still gets its own commit line first. The walk verifies each such record against the just-matched core beat and spike's resume pc.
4. **Termination.** The core's final beat is the ecall trap beat; spike continues with the ecall commit and `trap_user_ecall` at the halt PC, then the p-environment's write_tohost tail (length varies per run — not diffed). The exit code is the verdict.

19.3 ma_data, an expected halt

kestrel halts on a misaligned load or store (Chapter 5), and so does the pinned spike 1.1.0 in its own way: without the compile-time `RISCV_ENABLE_MISALIGNED` it traps them. On `rv32ui-p-ma_data` both stop at the same instruction, the first misaligned load: kestrel with cause 4, spike with `trap_load_address_misaligned` (and exit code 156, since the test never reaches its pass path). The battery lists `ma_data` as an expected halt: it passes when kestrel halts with cause 4 and the lockstep ends on that spike trap at the halt PC. Until 2026-10-20, when kestrel completed misaligned accesses in hardware, the battery used a side-by-side `--enable-misaligned` rebuild of the same spike commit instead.

19.4 The mnstatus divergence

Every test's p-environment preamble executes `csrwi mnstatus, 8` — a Smrnmi CSR, illegal in RV32I per spike's default extension set. Spike traps it and resumes at `mtvec`, which the p-environment points at the next instruction; kestrel's CSR stub simply retires it (Chapter 2). The 1:1 stream walk stays aligned because spike logs the trapping instruction's commit, the exception record is verified against the just-matched core beat and spike's resume pc, and the event is counted and reported as `1 stub trap(s) [trap_illegal_instruction]` per test. Any *other* mid-stream exception is a hard failure — none ever occurred.

Source: task-8 report (spike format notes 1-8, battery table); `dv/tbclasses/kestrel/rv32ui_battery.py` (spike runner, log parser, lockstep_diff)

20 The riscv-formal Flow

20.1 What it is

riscv-formal is the open-source RISC-V formal-verification framework: per-instruction *specification models* written as Verilog (a BEQ model, an LW model, …), paired with checkers that relate a core's RVFI beats to the model's expected behavior on every cycle, discharged by bounded model checking. Where simulation shows the programs you thought of behave, riscv-formal makes claims about *every* stimulus up to a proof depth — including the perverse ones no test author would write.

20.2 kestrel's setup

`formal/kestrel/riscv-formal/` holds a thin `rvfi_wrapper` around `kestrel_core` in the shape riscv-formal's reference cores use (picorv32's), a `checks.cfg`, and a Makefile driving genchecks and sby. The wrapper is worth understanding because it is small:

- `RISCV_FORMAL_ALIGNED_MEM` is defined (2026-10-20): memory is reported as a word address with byte-lane masks, and the instruction models require `rvfi_trap` on every misaligned load and store, which is kestrel's halt.
- Memory inputs (`imem_rdata`, `dmem_rdata`) are free `rvformal_rand_reg` values — there is no memory array. The checkers relate the core's *reported* rvfi_mem channels and writeback to the ISA model, so random read data is exactly the right stimulus, and no imem-stability assumption is needed even under free `mem_wait` stalls.
- The core lacks `rvfi_halt/rvfi_intr/rvfi_mode/rvfi_ixl` ports, so the wrapper generates them: `rvfi_halt = rvfi_valid & halt` (for a core with no trap handler, the Task-8 trap beat *is* the final retirement — precisely rvfi_halt semantics), `rvfi_intr = 0`, `rvfi_mode = M`, `rvfi_ixl = 32`. `RVFI_CONN` is not used.
- `RESET_ADDR` stays at the core default: a parameter is an elaboration constant and cannot be made symbolic without touching RTL; the checks are address-agnostic, so no proof is weakened.

20.3 The checks and depths

genchecks generates 42 checks from `isa_rv32i.txt`: the 37 instruction models plus five consistency checks — `reg` (register file integrity), `pc_fwd`/`pc_bwd` (PC monotonicity both directions), `liveness` (something eventually retires), and `unique` (no duplicate retirement orders). This pinned genchecks emits no `mem`/`causal`/`ill`/`hang`/ `cover` checks; load/store semantics are covered by the instruction models plus `reg`. Depths: instruction and consistency checks at 6 — enough for reset plus a few memory wait states with headroom — with liveness/unique at trigger 6, check 30 per the picorv32 configuration. Solver: z3 via smtbmc.

20.4 The flow's three honest workarounds

None weakens a proof; all are recorded because the next rung inherits the flow:

1. **sv2v pre-conversion.** This yosys (0.62+95) cannot parse the RTL's module-body `import rv32i_pkg::*;`, so the Makefile converts the RTL into plain Verilog before genchecks runs; the wrapper is still read as SystemVerilog.
2. **`const rand reg` workaround.** The same yosys rejects `const rand reg` (used by five consistency checkers), and this build has no `system` command to patch copies at run time. The Makefile therefore builds the checks directory as a symlink farm around a sed-patched *copy* of `rvfi_macros.vh` (`const rand reg` → `(* anyconst *) reg`, verified semantically identical); the vendor tree is untouched.
3. **genchecks' working-directory contract.** This pin's genchecks reads its config from the current directory and requires cwd to sit exactly two levels below the riscv-formal root; the Makefile runs it from `formal/kestrel/riscv-formal` accordingly.

20.5 Results

The first clean build proved the flow end to end and returned 34/42 passing — the eight failures all counterexamples of one genuine core bug, told in the next section. After the fix, a full rebuild reports **42/42 PASS** (z3/smtbmc) — and the pre-fix 34/42 failure had itself reproduced identically across three full runs including one from `make clean`, so the green verdict is stable state, not a lucky run:

```
export PATH=/mnt/data/tools/oss-cad-suite/bin:/tmp/sv2v/sv2v-Linux:
$PATH
make -C formal/kestrel/riscv-formal checks   # generate + run all 42
sby tasks
```

Per-check evidence lives in `checks/<check>/status` (PASS/FAIL with step counts) plus logs and VCD counterexamples under `checks/<check>/`.

Source: task-9 report and fix round 1 (check list, wrapper notes, flow deviations, depths, results); `formal/kestrel/riscv-formal/` (formal_kestrel.sv, checks.cfg, Makefile)

21 The CEX Story: What Formal Caught That Forty-Two Tests Missed

This section is the verification chapter's lesson, and it deserves its own space: the riscv-formal counterexamples found a real, architecturally visible core bug that the entire directed battery, two lockstep methods, and code review had all missed. The suite keeps this story on record because it is the strongest argument for the ladder's verification doctrine — simulation shows what you thought of; proof covers what you didn't.

21.1 The failure

The first full formal run returned **34/42 PASS, 8 FAIL** — the six branch models plus `jal` and `jalr`, every one counterexampled the same way at the same line: `rvfi_insn_check.sv:178`, `assert(spec_trap == trap)`, at check step 6. The evidence chain, shown for `insn_beq_ch0`, is identical for all eight:

1. sby: `failed assertion ... at rvfi_insn_check.sv:178.6 step 6`.
2. That line compares the model's expected trap against the core's reported `rvfi_trap`. The checker assumed a valid BEQ beat; the ISA model says it traps; the core reported no trap.
3. The VCD at the check step shows `spec_valid=1, spec_trap=1, rvfi_trap=0`, with retired instruction `0x980001E3` — a `beq x0, x0` (opcode `1100011`, funct3 `000`), taken by construction, whose B-immediate targets `0xFFFFF996`: an address with `next_pc[1:0] = 2`. (The sibling checks carry their own stimuli — `insn_bne_ch0`'s trace, for instance, retires `0x80121163`, a taken `bne` to another misaligned target.)
4. The model source (`vendor/riscv-formal/insns/insn_beq.v`) computes `spec_trap = (next_pc[1:0] != 0) || !misa_ok` for IALIGN=32: any taken control transfer to a non-4-aligned target must trap. The JAL and JALR models carry the identical condition.
5. rvfi.md is unambiguous: "`rvfi_trap` must also be set for … a jump instruction that jumps to a misaligned instruction."
6. The spec (Volume I, section 2.1.5) names the event: instruction-address-misaligned.

kestrel at the time silently jumped: `pc <= pc + imm` (or the JALR target masked to ~1) with `rvfi_trap = 0`, then fetched and executed at the misaligned address. A taken branch to address 0x2 executed garbage as if nothing were wrong.

21.2 Why the battery missed it

Forty-two directed tests, a golden interpreter, and a second emulator all passed — because not one of them ever targeted a misaligned instruction address. The rv32ui programs are linked code; every branch and jump target is assembler-controlled and 4-byte aligned by construction. The bug lived exactly where directed tests cannot reach: in the space of operands and targets no test author writes. Formal verification's contribution is not diligence but *coverage of the absurd*: the model checker gleefully retires BEQ with every immediate, including the ones that land on `next_pc[1:0] != 00`, and it found the hole on depth 6 of an unconstrained search.

21.3 The fix

The fix is small, follows the core's existing idiom, and is the HALT_IALIGN material of Chapter 5: `halt_cause` encoding moves into `rv32i_pkg` as the single source of truth with the new `HALT_IALIGN = 4'h3`; `kestrel_core` computes `misalign_target = (jump | jalr | branch_taken) & (next_pc[1:0] != 2'b00)` and OR-s it into the existing halt path — same holding register, same trap beat, same PC freeze. The one delicate point is the link register: decode keeps `rd_wen = 1` for JAL/JALR, so both the register write and the RVFI report are gated by `halt` itself, not by decode's `rd_wen`. The golden interpreter models the same halt (cause 3, `pc_wdata` equal to the misaligned target), and a directed program — `system_misalign_jmp` — pins it: an aligned JAL regression (link value `pc+4` retires normally), then `jal x0, .+2` must halt with cause 3, trap beat `pc_wdata = 0x12`, no register write.

21.4 Aftermath

Re-run against the patched RTL: riscv-formal **42/42 PASS**, the rv32ui battery **42/42 PASS** with spike lockstep intact (no rv32ui test targets a misaligned PC, so the battery stayed green as predicted), and every pre-existing regression — core, branch, load/store, decode suites at gate and func — green. The task's mandated commit message reported the intermediate 34/42 honestly rather than claiming green; the fix round is the commit that earns the claim.

21.5 The lesson, stated once

Each method in this chapter sees a different slice of behavior. The golden interpreter sees the same conventions the RTL was written with. Spike sees the same programs, independently implemented. The battery sees real compiled code. riscv-formal sees everything the ISA permits up to a depth — and at this core's scale (depth 6, minutes per check) that "everything" is cheap enough to run on every RTL change. The ladder's doctrine follows directly: directed tests for the common case, lockstep for independence, and formal proofs as the gate that keeps honest optimizations honest.

Source: task-9 report (counterexample chain, proposed fix) and fix round 1 (fix, directed test, re-run evidence); `common-rv32i/rtl/includes/rv32i_pkg.sv`, `rtl/top/kestrel_core.sv`; `dv/tests/programs/system_misalign_jmp.s`

22 What kestrel Costs

22.1 CPI 1, honestly stated

kestrel retires one instruction per clock — CPI 1 — for every instruction in the ISA. Two honesty clauses attach to that claim. First, behind block RAM a load occupies two clocks (the memory holds the core while its data read lands), so the average is 1 plus the load fraction; with combinational memories it is exactly 1. Second, CPI 1 is *per instruction*, and instructions are not the unit anyone asked for — programs are. The single-cycle discipline taxes the fast instructions to pay for the slow ones, and the tax collector is the clock period.

22.2 The critical path is the whole machine

In a single-cycle design, the clock period must span the slowest instruction's path through *every* block, because every block sits between the same two edges. Trace kestrel's worst case, an ALU instruction, and count what the clock must cover:

1. The PC register updates and presents `imem_addr`.
2. The instruction memory reads combinationally; the instruction word settles.
3. Decode (`{opcode, funct3, funct7}` case) and the immediate generator settle in parallel with the register file's combinational reads.
4. The source muxes feed the ALU; the ALU result settles. (For a load, the address then crosses the data memory, and the read data rotates and sign-extends into the writeback mux.)
5. The next-PC mux selects; for branches, the dedicated comparator's decision must also settle to drive that selection.
6. The writeback data reaches the register file's data input and the PC data reaches the PC register, both meeting setup before the edge.

Every one of those stages — two memories, a truth table, a register file, an ALU, three muxes, a comparator — is on the critical path for *some* instruction, so the clock is limited by their *sum*, not their slowest member. The ADD that could have run at the ALU's speed is clocked at the LW's speed, and both are clocked below what a leaner machine could do. Patterson and Séquin's RISC I made this trade deliberately in 1981: one instruction, one cycle, clock be damned, because a machine whose every instruction completes in one cycle is a machine you can reason about completely (papers, entry 1). Kestrel inherits the reasoning and, this chapter argues, should not inherit the clock.

22.3 The accounting, instruction by instruction

Critical-path content per instruction class

Instruction class	Path the clock must span
ALU (OP/OP-IMM)	imem + decode + regfile + ALU + writeback setup
Branch	imem + decode + regfile + comparator + next-PC mux + PC setup
Load	imem + decode + regfile + ALU + dmem + rotate/sign + writeback setup
Store	imem + decode + regfile + ALU + dmem + strobe setup
JAL/JALR	imem + decode + regfile (JALR) + add + next-PC mux + PC setup

The table's lesson: there is no clock frequency at which this machine is fast, only a frequency at which it is correct, and that frequency is set by the longest row — almost always the load, which crosses both memories and the datapath between them.

22.4 What would have to give

The single-cycle tax has exactly one root cause: every stage serves every instruction every cycle. Relax that and the frequency rises, but the reasoning burden rises with it — which is the trade the ladder is built to teach. The moment work is spread across multiple cycles, the stages need boundaries (pipeline registers), the boundaries create overlaps (hazards), and the hazards need policy (forwarding, stalls, flushes). That is not a defect of the next design; it is the price of the frequency, and merlin exists to teach how to pay it deliberately.

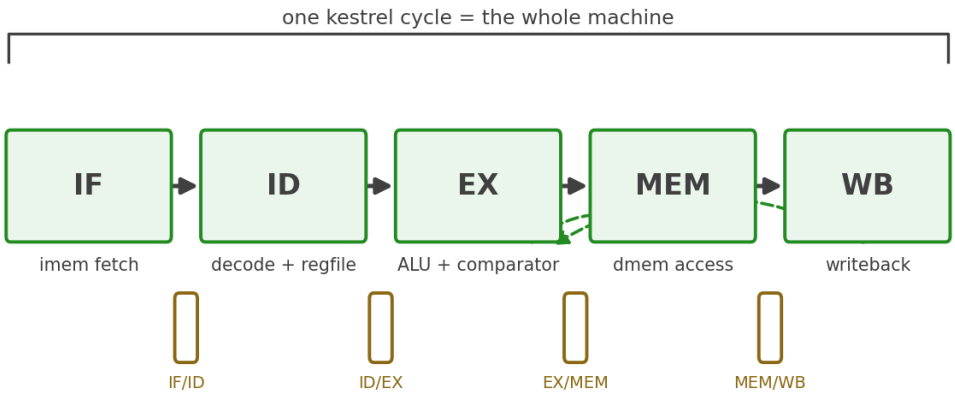
Source: `rtl/top/kestrel_core.sv` (datapath structure); Patterson and Séquin, “RISC I” (ISCA 1981) and Hennessy et al., “MIPS: A Microprocessor Architecture” (MICRO 1982) — papers entries 1 and 3; Flynn, “Very High-Speed Computing Systems” (1966) for the throughput-versus-clock framing (papers, entry 2)

23 The Next Rung: merlin

23.1 The five-stage answer

merlin, rung 2 of the falcon suite, keeps kestrel's ISA bit-for-bit and changes the time dimension: instead of one cycle per instruction, an instruction flows through five stages — IF, ID, EX, MEM, WB — with pipeline registers between them, so five instructions are in flight at once and the clock only has to span one stage. The same ADD that paid for the load's path in kestrel pays for the ALU's alone in merlin; that is the entire economic argument for pipelining, and it is also where every new problem comes from.

23.1.1 Figure 7.1: kestrel's cycle, cut into merlin's stages



From one kestrel cycle to merlin's five stages

The figure draws the correspondence explicitly: kestrel's combinational cloud between two PC edges is merlin's five stage boundaries. Nothing new is *computed* at rung 2; the computation is the same — it is *overlapped*.

23.2 What appears when the cycle is cut

Each new mechanism is the direct consequence of one cut:

- **Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB).** The boundary between stages becomes state, which is the first real state kestrel's control has ever had.
- **RAW hazards and forwarding.** An instruction in EX may need the result of the instruction ahead of it, still in MEM or WB. merlin forwards EX/MEM and MEM/WB results back into EX with an explicit priority — the forwarding matrix is Chapter 1 of merlin's book.
- **Load-use interlock.** A load's data isn't available until the end of MEM, one cycle too late for the dependent instruction in EX. The interlock stalls one cycle — the first stall the suite has ever had.
- **Control hazards.** Branches resolve in EX under merlin's predict-not-taken policy; a taken branch flushes the two fetched instructions behind it. Kestrel's taken-branch timing freedom (one cycle either way) becomes a two-instruction penalty and a flush path.
- **The same verification, adapted.** The RVFI aggregation that kestrel drives combinationally per cycle becomes merlin's retirement logic, and the riscv-formal checks are adapted to a retire interface; the battery, the lockstep harness, and the golden interpreter carry over because the ISA does not change.

23.3 What carries over unchanged

The investment kestrel already made transfers directly: the decode truth table (Chapter 4) is still the complete specification of what each instruction does — merlin just evaluates it in different stages; the register file and ALU leaves are reused as-is; the memory contract's word-wide, byte-strobed data port is unchanged; the halt causes survive as the core's stop contract; and the documentation style — decisions stated plainly, spec citations by chapter — is the suite's doctrine, not kestrel's habit. Even the CSR stub carries forward unchanged: merlin has no more CSR state than kestrel.

23.4 Why the ladder stops here first

kestrel is complete on its own terms — a verified RV32I core with a documented contract and a proof-backed verification record. What it is not is fast, and it cannot be made fast without becoming something else. The suite's argument is that "something else" is best understood as a sequence of one-concept-at-a-time changes, each motivated by a cost the previous rung made visible. Chapter 7's cost accounting is the motivation; merlin is the first payment. When its book exists, it will cite this one the same way this one cites the specification — by chapter, with the seams labeled.

Source: falcon-suite design spec, "Rung 2 — merlin"; Hennessy et al., "MIPS" (papers, entry 3) for the stage partition; Flynn (papers, entry 2) for throughput versus latency